Matriz de Evaluación y Análisis Comparativo de Configuraciones

1. Resumen Ejecutivo

Se realizaron **11 rondas de evaluación** con un total de ~**35 conversaciones** independientes, abarcando dos modelos LLM (Gemini Flash 3.1 Lite Preview y Llama 3.1 local), dos familias de modelos de embedding (Gemini Embedding 001 y BAAI/bge-m3), tres técnicas de chunking (fixed_size_overlap, recursive y paragraph_custom), tres estrategias de recuperación (similarity_search, MMR y threshold) y múltiples combinaciones de hiperparámetros.

El puntaje más bajo registrado fue **0/18 (0%)** con threshold=0.7, y el más alto fue **14/18 (77.8%)** con fixed_size_overlap, chunk_size=300, overlap=30 y similarity_search. La evolución del sistema muestra un progreso claro desde las primeras pruebas (~5/18) hasta las últimas (~11-14/18), impulsado principalmente por mejoras en el saneamiento del corpus y ajustes en la estrategia de chunking.

---

2. Tabla Maestra de Configuraciones y Resultados

2.1 Tests con Gemini Flash 3.1 Lite Preview + gemini-embedding-001

Test	Conv.	Chunking	Chunk Size	Overlap	Retrieval	top_k	Temp.	Punta
GeminiPt1	1	fixed_size_overlap	800	200	similarity_search	5	0.7	5.0
GeminiPt1	2	fixed_size_overlap	800	200	mmr	5	0.7	4.5
GeminiPt1	3	fixed_size_overlap	800	200	threshold (0.7)	5	0.7	**0.0**
GeminiPt1	4	fixed_size_overlap	800	200	threshold (0.5)	5	0.7	5.5
GeminiPt1b	1	fixed_size_overlap	800	200	threshold (0.5)	5	0.7	5.5
GeminiPt2	1	recursive	1024	50	similarity_search	7	0.7	1.0
GeminiPt2	2	recursive	1024	50	mmr	7	0.7	1.5
GeminiPt2	3	recursive	1024	50	threshold (0.5)	7	0.7	1.0
GeminiPt3	1	paragraph_custom	1024	50	similarity_search	7	0.7	**6.5**
GeminiPt3	2	paragraph_custom	1024	50	mmr	7	0.7	5.0
GeminiPt3	3	paragraph_custom	1024	50	threshold (0.5)	7	0.7	6.5

2.2 Test con Gemini 2.5 Flash + gemini-embedding-2-preview (interrumpido)

Test	Conv.	Chunking	Chunk Size	Overlap	Retrieval	top_k	Temp.	Puntaje
GeminiV2	1	fixed_size_overlap	300	30	similarity_search	5	1.0	N/A (8 preg.)

Nota: Este test fue interrumpido a las 8 preguntas por agotamiento de cuota (5 req/min en tier free de Gemini 2.5 Flash). En las preguntas evaluadas, las respuestas fueron precisas cuando el retrieval funcionó, pero los chunks de 300 caracteres resultaron demasiado fragmentados para capturar contexto completo (ej: TORA no pudo ser descrita completamente).

2.3 Tests con Llama 3.1 (Ollama) + BAAI/bge-m3 — Fase temprana (sin puntajes formales)

Test	Conv.	Chunking	Chunk Size	Overlap	Retrieval	top_k	Temp
Ollama-BGEm3	1	recursive	512	50	similarity_search	5	0.5
Ollama-BGEm3	2	recursive	512	50	similarity_search	5	1.0
Ollama-BGEm3	3	recursive	1000	200	similarity_search	5	1.5
Ollama-BGEm3	4	recursive	1000	200	similarity_search	5	2.0
Ollama-BGEm3	5	fixed_size_overlap	300	30	similarity_search	5	1.0
Ollama-BGEm3	6	fixed_size_overlap	300	30	mmr	8	1.0
Ollama-BGEm3	7	fixed_size_overlap	300	30	mmr	8	1.5
Ollama-BGEm3	8	recursive	600	100	mmr	8	2.0
Ollama_BGEm3-2	1-10	Múltiples	Múltiples	Múltiples	Múltiples	5-7	0.7

Nota: Estos tests tempranos sirvieron para identificar problemas críticos en el saneamiento (separadores \n \n en vez de \n\n) y para descartar temperaturas altas (≥1.5) por generar alucinaciones.

2.4 Tests con Llama 3.1 (Ollama) + BAAI/bge-m3 — Post-corrección de saneamiento

Test	Conv.	Chunking	Chunk Size	Overlap	Retrieval	top_k	Temp.	Punta
OllamaV2	1	recursive	512	100	similarity_search	5	1.0	11.0
OllamaV2	2	recursive	512	100	mmr	8	1.0	10.5
OllamaV2	3	fixed_size_overlap	300	30	similarity_search	5	1.0	11.5
OllamaV2	4	fixed_size_overlap	300	30	mmr	8	1.0	8.5
OllamaV2	5	recursive	500	100	mmr	8	1.0	10.5
OllamaV2	6	recursive	500	100	similarity_search	5	1.0	—
OllamaV3	1	recursive	512	100	similarity_search	5	1.0	10.0
OllamaV3	2	recursive	512	100	mmr	8	1.0	9.0
OllamaV3	3	fixed_size_overlap	300	50	similarity_search	5	1.0	10.0
OllamaV3	4	fixed_size_overlap	300	50	mmr	8	1.0	8.5
OllamaV3	5	recursive	500	100	mmr	8	1.0	9.5
OllamaV3	6	recursive	600	100	mmr	8	1.0	10.0
OllamaV4	**1**	**fixed_size_overlap**	**300**	**30**	**similarity_search**	**5**	**1.0**	**14.0**
OllamaV4	2	recursive	512	100	similarity_search	5	1.0	13.5
OllamaV4	3	recursive	500	100	mmr	8	1.0	12.5
OllamaV5	1	fixed_size_overlap	300	30	similarity_search	5	1.0	11.5
OllamaV5	2	recursive	512	100	similarity_search	5	1.0	10.0
OllamaV5	3	recursive	500	100	mmr	8	1.0	11.0
OllamaV5	4	recursive	300	50	similarity_search	5	1.0	11.0

3. Análisis Comparativo por Variable

3.1 Impacto de la Técnica de Chunking

La técnica de chunking resultó ser la variable con mayor impacto en la calidad de recuperación.

recursive (chunk_size=1024, Gemini): Generó el peor rendimiento registrado entre las técnicas (1.0-1.5/18). La conclusión del test fue contundente: los fragmentos perdieron completitud semántica, produciendo chunks como "Promoción:", "Bases de Datos:" o "Investigación Operativa." sin contexto útil. El buscador semántico no logra encontrar información relevante en fragmentos tan desarticulados.

paragraph_custom (chunk_size=1024, Gemini): Demostró ser superior en la conservación de significado semántico (5.0-6.5/18). Al tomar el párrafo como unidad base, los chunks mantienen coherencia temática. Este fue el mejor resultado en la fase Gemini.

fixed_size_overlap (chunk_size=800, Gemini): Rendimiento intermedio (4.5-5.5/18), mejor que recursive pero inferior a paragraph_custom con el mismo modelo.

fixed_size_overlap (chunk_size=300, Ollama post-fix): Sorprendentemente, esta configuración produjo el mejor resultado absoluto (14.0/18). La hipótesis es que los chunks pequeños con el embedding local bge-m3 permiten un matching semántico más preciso, siempre que el saneamiento sea correcto.

Conclusión: No existe una técnica universalmente superior. La interacción entre técnica de chunking, tamaño del chunk, modelo de embedding y calidad del saneamiento determina el resultado.

3.2 Impacto de la Estrategia de Recuperación

Estrategia	Promedio Gemini	Promedio Ollama (post-fix)	Observación
similarity_search	3.9/18	**11.3/18**	Consistentemente la mejor o segunda mejor
mmr	3.3/18	9.9/18	Tiende a perder ~1-2 puntos respecto a similarity
threshold (0.5)	3.5/18	—	Similar a similarity pero con riesgo de perder chunks
threshold (0.7)	**0.0/18**	—	Rompe completamente el sistema

similarity_search resultó ser la estrategia más confiable. Si bien MMR intenta diversificar resultados y evitar redundancia, en la práctica pierde chunks relevantes que sí comparten similitud temática. threshold con umbral 0.7 demostró ser catastrófico: ningún chunk alcanzó ese nivel de similitud, dejando al LLM sin contexto alguno. Con umbral 0.5, el rendimiento es comparable a similarity_search.

3.3 Impacto del Modelo LLM + Embedding

Combinación	Mejor Puntaje	Promedio	Tokens promedio
Gemini Flash 3.1 Lite + gemini-embedding-001	6.5/18 (36.1%)	~3.6/18	600-700 por consulta
Llama 3.1 (Ollama) + BAAI/bge-m3 (pre-fix)	Sin puntaje formal	~3.6/18 (estimado)	Mayor (local)
Llama 3.1 (Ollama) + BAAI/bge-m3 (post-fix)	**14.0/18 (77.8%)**	~10.7/18	Mayor (local)

La diferencia drástica entre las fases pre-fix y post-fix de Ollama demuestra que la calidad del saneamiento era el cuello de botella principal, no el modelo en sí. Con el corpus correctamente saneado, la combinación local (Llama 3.1 + bge-m3) superó ampliamente al pipeline en la nube (Gemini).

Es importante aclarar que los tests de Gemini se realizaron antes de la corrección del saneamiento, por lo que la comparación directa no es del todo justa. No obstante, Gemini mostró un excelente comportamiento de mitigación de alucinaciones: siempre que el contexto era insuficiente, se abstuvo de inventar información.

3.4 Impacto de la Temperatura

Temperatura	Comportamiento Observado
0.5	Respuestas concisas, a veces demasiado escuetas
0.7	Buen equilibrio en Gemini; usado en todos los tests de Gemini
1.0	Usado en todos los tests de Ollama post-fix; buen equilibrio
1.5	Empieza a generar alucinaciones y divagaciones
2.0	Alucinaciones severas; el modelo inventa información

Las temperaturas entre 0.7 y 1.0 demostraron ser el rango óptimo para este caso de uso RAG.

3.5 Impacto del top_k

top_k	Promedio observado	Nota
5	~11.0/18 (Ollama post-fix)	Suficiente para la mayoría de preguntas
7	Variable	Usado con Gemini; no mejoró respecto a 5
8	~9.5/18 (Ollama post-fix)	Generalmente usado con MMR; el resultado menor se debe más a MMR que a top_k

Un top_k de 5 resultó suficiente y más eficiente. Aumentar a 7-8 no mejoró resultados y puede introducir ruido.

---

4. Evolución del Sistema: Antes vs. Después

4.1 Fase Inicial (Ollama-BGEm3, GeminiPt1)

- **Puntaje típico:** 0-5.5/18 (0%-30.6%)
- **Problema principal:** Saneamiento deficiente. Los archivos .toon contenían separadores \n \n en lugar de \n\n, lo que impedía que `chunk_paragraph_custom()` reconociera los límites entre párrafos. Además, URLs pegadas al texto (ej: "Telegramhttps://...") contaminaban los embeddings.
- **Comportamiento:** El LLM recibía chunks descontextualizados o con información incompleta. Ante la falta de evidencia, el modelo correctamente se abstenía de responder (buen comportamiento de "no encuentro evidencia").

4.2 Fase de Diagnóstico (GeminiPt2, GeminiPt3)

- **Descubrimiento clave:** recursive con chunk_size=1024 destruía la semántica de los documentos (1.0/18). paragraph_custom con el mismo tamaño la preservaba mucho mejor (6.5/18).
- **Conclusión intermedia:** La técnica paragraph_custom es superior a recursive para estos documentos académicos con estructura de párrafos bien definida.

4.3 Fase Post-Corrección (OllamaV2-V5)

- **Puntaje típico:** 8.5-14.0/18 (47.2%-77.8%)
- **Mejoras aplicadas:** Corrección del saneamiento (normalización de líneas en blanco), mejora del overlap para preservar continuidad contextual, y ajustes al prompt del sistema.
- **Resultado:** El salto de ~3.6/18 promedio a ~10.7/18 promedio demuestra que la calidad del preprocesamiento es el factor más determinante del rendimiento de un RAG.

4.4 Línea de Tiempo de Puntajes (mejores por ronda)

```
Ollama-BGEm3:      ~3.6/18  (estimado, pre-fix)
GeminiPt1:          5.5/18  (30.6%)
GeminiPt2:          1.5/18  (8.3%)  ← recursive destruye
contexto
GeminiPt3:          6.5/18  (36.1%) ← paragraph_custom mejora
OllamaV2:          11.5/18  (63.9%) ← POST-FIX saneamiento
OllamaV3:          10.0/18  (55.6%)
OllamaV4:          14.0/18  (77.8%) ← MEJOR RESULTADO
OllamaV5:          11.5/18  (63.9%)
```

5. Las 5 Mejores Configuraciones

Ranking	Test	Chunking	Chunk Size	Overlap	Retrieval	top_k	Temp.	Pun
1°	OllamaV4-C1	fixed_size_overlap	300	30	similarity_search	5	1.0	**14.0**
2°	OllamaV4-C2	recursive	512	100	similarity_search	5	1.0	**13.5**
3°	OllamaV4-C3	recursive	500	100	mmr	8	1.0	**12.5**
4°	OllamaV2-C3	fixed_size_overlap	300	30	similarity_search	5	1.0	**11.5**
4°	OllamaV5-C1	fixed_size_overlap	300	30	similarity_search	5	1.0	**11.5**

Patrón común de las mejores configuraciones:

- Modelo de embedding: BAAI/bge-m3 (local)
- LLM: Llama 3.1 (local vía Ollama)
- Corpus: Post-corrección de saneamiento
- Estrategia de recuperación: similarity_search preferida sobre MMR
- top_k: 5
- Temperatura: 1.0
- Chunk size: 300-512 (relativamente pequeño)

---

6. Las 5 Peores Configuraciones

Ranking	Test	Chunking	Chunk Size	Overlap	Retrieval	top_k	Temp.	Pur
18°	GeminiPt1-C3	fixed_size_overlap	800	200	threshold (0.7)	5	0.7	**0.0**
17°	GeminiPt2-C1	recursive	1024	50	similarity_search	7	0.7	**1.0**
16°	GeminiPt2-C3	recursive	1024	50	threshold (0.5)	7	0.7	**1.0**
15°	GeminiPt2-C2	recursive	1024	50	mmr	7	0.7	**1.5**
14°	GeminiPt1-C2	fixed_size_overlap	800	200	mmr	5	0.7	**4.5**

7. Análisis por Tipo de Pregunta

7.1 Preguntas de Recuperación Directa (P1-P6)

Las preguntas directas como "¿Para qué sirve el software TORA?" fueron las mejor resueltas en todas las configuraciones. Incluso en las peores configuraciones (GeminiPt1), el sistema acertaba 2-3 de estas 6 preguntas. En las mejores configuraciones (OllamaV4), se acertaban las 6.

Preguntas consistentemente acertadas: P4 (regularidad en Comunicaciones y Redes) y P6 (software TORA) fueron las más estables, probablemente porque los chunks relevantes contenían palabras clave muy específicas que facilitaban el matching semántico.

Preguntas problemáticas: P3 (plan de estudios de Metodología) falló frecuentemente porque el LLM interpretaba "plan de estudios" de forma literal y no lo asociaba con "programa de contenidos", que es el término usado en el documento fuente.

7.2 Preguntas de Integración (P7-P12)

Estas preguntas requieren combinar información de múltiples documentos. La P8 (impacto de IA en asignaturas) fue consistentemente bien resuelta porque el embedding capturaba la semántica de "inteligencia artificial" y la cruzaba con el documento de Informática y RRHH. La P9 (comparación de requisitos BDA vs Redes) falló frecuentemente porque el chunk con los requisitos de BDA rara vez aparecía en los top-k.

7.3 Preguntas de Borde (P13-P18)

Las preguntas trampa fueron, paradójicamente, las mejor manejadas. El sistema consistentemente respondía "No dispongo de información en mis documentos" para P13 (Cálculo 1er año), P15 (costo certificación Cisco), P16 (nota numérica de TPs), P17 (Simplex en Excel) y P18 (hora del parcial). La P14 (Schmukler en IA) fue la más interesante: el sistema correctamente identificaba su presencia en BDA y Redes, y señalaba la ausencia de evidencia para IA.

---

8. Hallazgos Clave

1. **El saneamiento del corpus es el factor más determinante.** La corrección de los separadores de párrafo elevó los puntajes de ~3.6/18 a ~10.7/18 promedio, un salto del 200% sin cambiar ningún hiperparámetro del RAG.
2. **threshold con umbral alto es destructivo.** Un threshold de 0.7 produjo 0/18, mientras que 0.5 fue comparable a similarity_search. El umbral debe ser conservador (≤0.5) o no usarse.
3. **similarity_search supera a MMR consistentemente** en 1-2 puntos. MMR sacrifica relevancia por diversidad, lo cual no beneficia a este caso de uso donde los chunks relevantes suelen provenir del mismo documento.
4. **Chunks pequeños (300-512) funcionan mejor que grandes (800-1024)** con el embedding bge-m3, posiblemente porque los vectores capturan mejor la semántica de fragmentos concisos.
5. **El modelo local (Llama 3.1 + bge-m3) superó al modelo en la nube (Gemini)** post-corrección. Aunque la comparación no es perfectamente justa (el saneamiento se corrigió entre ambas fases), los resultados sugieren que un embedding multilingüe especializado (bge-m3) puede superar a embeddings genéricos en dominios en español.
6. **La temperatura óptima está entre 0.7 y 1.0.** Valores ≥1.5 generan alucinaciones que contradicen el propósito del RAG.
7. **El LLM sabe decir "no sé".** Tanto Gemini como Llama 3.1 demostraron un excelente comportamiento de abstención ante evidencia insuficiente, validando la efectividad del system prompt diseñado.

---

9. Recomendaciones para Configuración Óptima

Basándonos en la evidencia experimental, la configuración recomendada es:

Parámetro	Valor Recomendado	Justificación
chunking_technique	fixed_size_overlap	Mejor puntaje absoluto (14/18)
chunk_size	300	Matching semántico más preciso con bge-m3
chunk_overlap	30-50	Suficiente para preservar contexto entre chunks
embedding_model	BAAI/bge-m3	Mejor rendimiento con textos en español
retrieval_type	similarity_search	Consistentemente superior a MMR y threshold
top_k	5	Suficiente y eficiente
max_context_chunks	5	Balance entre contexto y eficiencia de tokens
temperatura	0.7-1.0	Rango óptimo sin alucinaciones
llm_model	llama3.1	Mejor rendimiento post-fix